

CareerLaunch

Week-by-Week Build Guide

Reproduce the whole Cohort Enrolment & Student Success application yourself — one week at a time. Each week gives you: the **exact prompt** to have Claude generate that week's code, the **copy-and-paste commands** to deploy, test, commit and push it, and the **manual click-path steps** to build the same things by hand.

Reference repo: github.com/dangutdavid/oxfodabledeveloper1_training

Reference org: maren.dangut@gmail.com · **Version:** 2.0 · August 2026

How to use this guide: you will NOT clone the reference repo. You create your **own** GitHub repository and your **own** Developer org, then work through Part 1 (Week 1), Part 2 (Week 2) and Part 3 (Week 3). Each part starts with the AI prompt that generates that week's code, then the commands to deploy → test → commit → push, then the click-path to build the same things by hand. Replace every `<angle-bracket>` placeholder with your own value.

Placeholder	Replace with	Example used in the reference build
<code><your-github-username></code>	Your GitHub username	dangutdavid
<code><your-repo-name></code>	Your new repository name	oxfodabledeveloper1_training
<code><your-org-username></code>	Your Developer Edition org username	maren.dangut@gmail.com
<code><your-org-alias></code>	A short nickname for the org	devorg

Part 0 · One-time setup (before Week 1)

1. Sign up for a free Developer Edition org: developer.salesforce.com/signup — verify the email, set a password, note your org username.
2. Install: Google Chrome · VS Code + Salesforce Extension Pack · Salesforce CLI · Git · GitHub CLI (cli.github.com).
3. In the org: Setup → search **Deliverability** → Access level **All Email** (needed for the Week 2 welcome email).

Sign in and create your own empty repository (copy & paste):

```
# check the tools
sf --version
git --version
gh --version

# sign in to GitHub (one time)
gh auth login

# authorise your Salesforce org (opens a browser)
sf org login web --alias <your-org-alias>
sf org list

# create YOUR OWN empty repository
```

```
gh repo create <your-github-username>/<your-repo-name> --public  
gh repo view  <your-github-username>/<your-repo-name>
```

Discovery & First Build — the Course object

This week you build: the SFDX project · the `Course__c` object with 5 fields and a tab · two Course records · a grouped report.

W1-A · The Week 1 prompt — have Claude generate it

Open Claude Code in your working folder. Attach **one document only**:

`Oxfordable_Salesforce_Participant_Workbook_Week1.pdf`. Then paste, with placeholders filled in:

PROMPT — WEEK 1

Read the attached Week 1 workbook. Create a new Salesforce DX project named `careerlaunch` (sf project generate). Build Lab 1 as source metadata in `force-app/main/default`:

1. Custom object `Course__c` – label `Course`, plural `Courses`, Record Name "Course Name" (Text), enable Reports, Activities and Field History Tracking.
2. Five custom fields on `Course__c`:
 - `Course_Level__c` Picklist with values: `Introductory`, `Advanced`
 - `Duration_Weeks__c` `Number(2,0)`
 - `Fee__c` `Currency(10,2)`
 - `Active__c` `Checkbox`, default checked
 - `Description__c` `Long Text Area(1000)`
3. A `Courses` custom tab (any motif).

Deploy it to my org `<your-org-username>` and verify the deployment succeeded.

Then create these two Course records in the org:

"Salesforce Developer – Introductory"	Level Introductory	Duration 3	Fee 500	Active
"Salesforce Developer – Advanced"	Level Advanced	Duration 10	Fee 1500	Active

Finally: `git init`, commit everything with message "Week 1: Course object, tab and records", and push to `https://github.com/<your-github-username>/<your-repo-name>` on branch `main`.

W1-B · Or build it by hand (click path)

Create the object

1. Log in → gear icon → **Setup** → **Object Manager** → **Create ▼** → **Custom Object**.
2. Label **Course**, Plural **Courses**, Record Name **Course Name** (Text). Tick Allow Reports · Allow Activities · Track Field History · **Launch New Custom Tab Wizard after saving** → Save → pick a tab style → Next → Next → Save.

Add the fields

Object Manager → Course → **Fields & Relationships** → **New** (use Save & New between fields):

Field Label	Data Type	Settings	API name created
Course Level	Picklist	Values one per line: Introductory, Advanced	Course_Level__c
Duration (Weeks)	Number	Length 2, Decimal Places 0	Duration_Weeks__c
Fee	Currency	Length 10, Decimal Places 2	Fee__c
Active	Checkbox	Default: Checked	Active__c
Description	Text Area (Long)	Length 1000	Description__c

Create the two records

App Launcher → **Courses** → **New**:

Course Name	Course Level	Duration	Fee	Active
Salesforce Developer — Introductory	Introductory	3	500	Checked
Salesforce Developer — Advanced	Advanced	10	1500	Checked

Build the report

App Launcher → **Reports** → **New Report** → Report Type **Courses** → Start Report → drag **Course Level** into **Group Rows** → **Run** → **Save as** `Courses by Level`. (Reports are built in the UI — not part of the CLI deployment.)

W1-C · Week 1 commands (copy & paste)

```
# verify (or perform) the deployment
cd careerlaunch
sf project deploy start --source-dir force-app --target-org <your-org-alias> --wait 15
sf org open --target-org <your-org-alias>           # check the Courses tab and records

# commit and push Week 1
git init
git add .
git commit -m "Week 1: Course object, tab and records"
git branch -M main
git remote add origin https://github.com/<your-github-username>/<your-repo-name>.git
git push -u origin main
```

Week 1 checkpoint: the Courses tab exists, both records are saved, the report groups them by level, and your repo's `main` branch holds the project.

Design & Build — Data model, Flows and Apex

This week you build: Cohort__c and Enrolment__c with relationships, roll-up, formulas and 2 validation rules · Contact.Total_Enrolments__c · 3 flows · the trigger, handler, Queueable and two invocable Apex services · sample data.

W2-A · The Week 2 prompt — have Claude generate it

In the same project folder, attach **one document only**: Oxfordable_Salesforce_Participant_Workbook_Week2.pdf .
Then paste:

PROMPT — WEEK 2

Read the attached Week 2 workbook. Extend my existing careerlaunch SFDX project with the full Lab 2-4 build, exactly matching the workbook's solution design (British spelling Enrolment__c with one "l" everywhere):

DATA MODEL (Lab 2)

1. Cohort__c – Record Name Auto Number CO-{0000}; fields: Course__c (required Lookup to Course__c), Start_Date__c (Date, required), End_Date__c (Date), Delivery_Mode__c (Picklist: Online, In-Person, Hybrid), Capacity__c (Number(3,0), required, default 20), Status__c (Picklist: Planned, Open, Full, Running, Completed, Cancelled), Enrolled_Count__c (Roll-Up Summary: COUNT of Enrolments where Status not equal to Cancelled), Seats_Remaining__c (Number formula: Capacity__c – Enrolled_Count__c), Cohort_Label__c (Text formula: Course__r.Name & " - " & TEXT(Start_Date__c)).
Validation rule End_Date_After_Start_Date:
AND(NOT(ISBLANK(End_Date__c)), End_Date__c < Start_Date__c)
with error "End Date cannot be earlier than Start Date."
2. Enrolment__c – Record Name Auto Number ENR-{00000}; fields: Cohort__c (Master-Detail to Cohort__c), Student__c (required Lookup to Contact), Status__c (Picklist: Applied default, Confirmed, Waitlisted, Cancelled, Completed), Enrolment_Date__c (Date), Amount_Paid__c (Currency(10,2), default 0), Enrolment_Key__c (Text 64, External ID, Unique), Attendance_Percent__c (Percent(3,0)), Certificate_Issued__c (Checkbox).
Validation rule Payment_Not_Above_Fee: Amount_Paid__c > Cohort__r.Course__r.Fee__c
with error "Amount Paid cannot exceed the course fee."
3. Contact.Total_Enrolments__c Number(3,0). Custom tabs for Cohort and Enrolment.

FLows (Lab 3)

4. Enrolment_Confirmed_Actions – record-triggered after-save on Enrolment__c, created or updated, entry condition Status__c = Confirmed only when updated to meet it. Elements: Get the parent Cohort; Send Email to {!\$Record.Student__r.Email} with subject "Your place is confirmed – {!\$Record.Cohort__r.Cohort_Label__c}" and a welcome body (put a FAULT PATH on the email action so a failed email never blocks the save); Create a Task "Send welcome pack to {!\$Record.Student__r.Name}" owned by the Cohort's owner, due today, priority High; Decision: Seats_Remaining <= 0; on Yes update the Cohort Status to Full. Deploy it Active.
5. Quick_Enrol_Student – screen flow: course picklist (record choice set on active Courses) -> cohort radio buttons (record choice set on Open cohorts of that course, labelled by Cohort_Label__c) -> student Lookup component (Enrolment__c.Student__c) -> Get the selected Course -> Apex action "Find Available Cohorts" -> decision on seatsAvailable -> create the Enrolment as Confirmed -> success screen; a "cohort just filled up" screen on No; fault connectors from every Get/Create/Action element to an error screen showing {!\$Flow.FaultMessage}. Deploy it Active.
6. Cohort_Start_Reminder – schedule-triggered daily at 07:00: get Confirmed enrolments, loop with a decision Cohort__r.Start_Date__c = {!\$Flow.CurrentDate}+3 (Date formula), build reminder Tasks into a collection, one Create Records after the loop (no DML in the loop). Deploy it Active.

APEX (Lab 4) – bulk-safe, no SOQL or DML inside loops, human-readable errors

7. EnrolmentTrigger (before insert/update, after insert/update) routing only, plus EnrolmentTriggerHandler with methods: setDefaults (Enrolment_Date today, Status Applied, Amount_Paid 0), buildUniqueKey (Enrolment_Key__c = first 15 chars of CohortId + "-" + first 15 of StudentId), preventOverbooking (count newly-consuming seats per cohort, ONE query for the cohorts, allocate seats in memory, addError "This cohort is full. Please choose another cohort or add the student to the waiting list." on records over capacity), enqueueStudentSummary (enqueue the Queueable, respecting the queueable job limit).
8. StudentSummaryQueueable – aggregate non-cancelled enrolments per student with one GROUP BY query and write Contact.Total_Enrolments__c (zero everyone first so cancellations are reflected).
9. CohortAvailabilityService – @InvocableMethod "Find Available Cohorts": input courseName; query Open future cohorts whose Course name matches LIKE; keep those with Seats_Remaining > 0; outputs: answer (human-readable summary), cohorts (list), seatsAvailable (boolean). Write a clear description on every @InvocableVariable.
10. EnrolmentStatusService – @InvocableMethod "Check Enrolment Status": input studentEmail; return the student's enrolments (cohort label, start date, status) as an answer string plus a found boolean.

Also write scripts/apex/seed.apex creating: cohort C0-0001 (Introductory course, starts today+14, capacity 3, Open), C0-0002 (Advanced, today+30, capacity 10, Open), and 3 student Contacts with email addresses.

Deploy everything to <your-org-username>, run the seed script, then commit with message "Week 2: data model, flows, Apex" and push to <https://github.com/<your-github-username>/<your-repo-name>>.

W2-B · Or build it by hand (click path)

Cohort__c

1. Object Manager → Create ▼ → Custom Object: Label **Cohort**, Record Name **Cohort Number**, Auto Number **C0-0000**, start 1, tick Reports/Activities/Tab Wizard.
2. Fields: Course (Lookup → Course, required) · Start Date (Date, required) · End Date (Date) · Delivery Mode (Picklist Online, In-Person, Hybrid) · Capacity (Number 3,0, default 20, required) · Status (Picklist Planned, Open, Full, Running, Completed, Cancelled).
3. **Wait** with Enrolled Count / Seats Remaining / Cohort Label until the master-detail exists (next step).

Enrolment__c (one "l" — British spelling, or the Apex will not compile)

1. Custom Object: Label **Enrolment**, Record Name **Enrolment Number**, Auto Number **ENR-000000**.
2. Fields **in this order**: **Cohort — Master-Detail** → **Cohort** (first!) · Student (Lookup → Contact, required) · Status (Picklist, **Applied default**, Confirmed, Waitlisted, Cancelled, Completed) · Enrolment Date (Date) · Amount Paid (Currency 10,2, default 0) · Enrolment Key (Text 64, tick **External ID + Unique**) · Attendance % (Percent 3,0) · Certificate Issued (Checkbox).

Back on Cohort — the three calculated fields

Field	Type	Definition
Enrolled Count	Roll-Up Summary	Enrolments · COUNT · Filter: Status not equal to Cancelled
Seats Remaining	Formula → Number (0 dp)	Capacity__c - Enrolled_Count__c
Cohort Label	Formula → Text	Course__r.Name & " - " & TEXT(Start_Date__c)

Validation rules + Contact field

- Cohort → Validation Rules → New → `End_Date_After_Start_Date : AND(NOT(ISBLANK(End_Date__c)), End_Date__c < Start_Date__c)` — “End Date cannot be earlier than Start Date.”
- Enrolment → `Payment_Not_Above_Fee : Amount_Paid__c > Cohort__r.Course__r.Fee__c` — “Amount Paid cannot exceed the course fee.”
- Contact → New field → Number(3,0), Label **Total Enrolments**.
- Tidy the Cohort page layout (add the 3 calculated fields + the Enrolments related list); create the sample data by hand or with the seed script.

Checkpoint (US-03): create one Enrolment linking a Contact to CO-0001 → Enrolled Count = 1, Seats Remaining = 2, with no automation at all.

The three flows and the Apex

Build the flows element-by-element exactly as itemised in the Week 2 prompt (points 4–6): Setup → Flows → New Flow → Record-Triggered / Screen / Schedule-Triggered — and **Activate each one**. Type the Apex (points 7–10) in Developer Console or VS Code; the full reference source is in the reference repo under `force-app/main/default/classes`.

Checkpoint (US-04/US-05): a 4th student on capacity-3 CO-0001 is blocked with your “cohort is full” message; the same student twice is blocked by the unique key. **Checkpoint (US-06/US-07):** setting Status to Confirmed creates a Task, and filling the last seat flips the Cohort to Full.

W2-C · Week 2 commands (copy & paste)

```
cd careerlaunch
# deploy the new metadata
sf project deploy start --source-dir force-app --target-org <your-org-alias> --wait 15

# seed the sample data
sf apex run --file scripts/apex/seed.apex --target-org <your-org-alias>

# commit and push Week 2
git add .
git commit -m "Week 2: data model, flows, Apex"
git push
```

Build, Test, Deploy & Demo — UI, AI, Tests and Release

This week you build: two LWC Apex controllers · three LWCs · the Enrolment Console app page · the Agentforce agent · the test suite (75%+ coverage) · the permission set · the release.

W3-A · The Week 3 prompt — have Claude generate it

Attach **one document only**: `Oxfordable_Salesforce_Participant_Workbook_Week3.pdf`. Then paste:

PROMPT — WEEK 3

Read the attached Week 3 workbook. Finish my careerlaunch SFDX project with Labs 5–8:

LWC + CONTROLLERS (Lab 5)

1. CohortController – @AuraEnabled(cacheable=true) getActiveCourses() and getOpenCohorts(Id courseId): Open, future-dated cohorts, optionally filtered by course, WITH SECURITY_ENFORCED, ordered by Start Date.
2. EnrolmentController – @AuraEnabled createEnrolment(cohortId, studentId, status): insert as Confirmed inside try/catch; on DmlException rethrow AuraHandledException with a friendly sentence (translate "duplicate value found" into "That student is already enrolled on this cohort."); return the saved record with its Name.
3. LWC cohortExplorer – course combobox + lightning-datatable of open cohorts (Cohort, Course, Starts, Mode, Capacity, Seats left) via @wire; row selection dispatches a "cohortselected" CustomEvent; public @api refresh() using refreshApex.
4. LWC enrolmentPanel – receives the cohort via @api; lightning-record-picker for the Contact; imperative call to createEnrolment; success toast; "enrolled" event up; NavigationMixin to the new record; error toast with the friendly message.
5. LWC enrolmentConsole – wrapper wiring the two together (data down via cohort={selectedCohort}, events up calling explorer.refresh()), exposed to lightning__AppPage.
6. A FlexiPage app page named "Enrolment_Console" containing enrolmentConsole.

TESTS (Lab 7) – all data created in-test, no hard-coded IDs, meaningful asserts

7. TestDataFactory – createCourse, createCohort, createStudents (insert Contacts with Database.DMLOptions DuplicateRuleHeader.allowSave = true so org duplicate rules never break tests), buildEnrolments (returned NOT inserted – the caller decides).
8. EnrolmentTriggerTest – testDefaultsApplied; testOverbookingBlocked (negative test with Database.insert(record, false)); testBulkInsertRespectsCapacity (200 records loaded as Applied against capacity 150 – assert the cohort NEVER exceeds capacity and the 50 overflow records are blocked); testDuplicateEnrolmentBlocked; testQueueableUpdatesContact (Test.startTest/stopTest forces the async job).
9. CohortServicesTest – both controllers and both invocable services, happy and error paths.

SECURITY + RELEASE (Lab 8)

10. Permission set Enrolment_Console_User: Read/Create/Edit on Course__c, Cohort__c and Enrolment__c; Read on Contact; field permissions on all custom fields (formulas and the roll-up read-only); Apex class access to the four non-test classes; access to the Quick_Enrol_Student flow; the three tabs Visible.
11. manifest/package.xml covering everything built across all three weeks.

Then: deploy to <your-org-username>, assign the permission set to my user, run ALL the Apex tests with coverage and fix any failures, and commit with message "Week 3: LWC console, tests, permission set" and push to <https://github.com/<your-github-username>/<your-repo-name>>. Report the test results.

W3-B · Manual steps you must finish in the org (click path)

Activate the Enrolment Console page (US-10)

1. Setup → **Lightning App Builder** → open **Enrolment Console** → **Activation...** → Lightning Experience tab → add to an app (e.g. Sales) → Save.
2. App Launcher → Enrolment Console → filter by course, select a cohort, pick a student, **Enrol student** → a toast appears, the seat count drops, and an overbooking attempt is still blocked by the Week 2 trigger.

Build the Agentforce agent (US-11, US-12) — Setup only, not deployable by CLI

1. Setup → **Einstein Setup** → turn on → Setup → **Agentforce Agents** (assign yourself the Agentforce permission set if prompted).
2. **New Agent**: type Employee/internal · Name **Oxfordable Student Assistant** · Role “Answers questions about our courses, cohort availability and a student's own enrolment status.” · Company “Oxfordable Careers, a technology training provider.” · Tone: warm, concise, professional — never invent dates, prices or seat numbers.
3. **Two actions from your invocable Apex**: Find Available Cohorts (input Course Name) and Check Enrolment Status (input Student Email). Write every input/output description for a colleague who has never seen your org — thin descriptions are the number-one reason agents misbehave.
4. **Topic “Course Enrolment Enquiries”** — classification: “Use this topic when someone asks about our courses, when a cohort starts, whether seats are available, how to join, or the status of an enrolment they already hold.” Scope: answer only from the assigned actions. Instructions: confirm the course before searching · always state start date and seats remaining · offer the waiting list when full · ask for the booking email for status questions · never share one student's details with another · pass anything else to the team. Assign both actions.
5. **Test in the preview**: “Do you have any Advanced cohorts with space?” → live list · “What is the status of my enrolment?” → asks for the email · an off-topic question → graceful decline. Read the **reasoning trace** after each test. Then **Activate**.

W3-C · Week 3 commands (copy & paste)

```
cd careerlaunch
# deploy
sf project deploy start --source-dir force-app --target-org <your-org-alias> --wait 15

# assign the permission set BEFORE running tests
# (WITH SECURITY_ENFORCED enforces the running user's field-level security)
sf org assign permset --name Enrolment_Console_User --target-org <your-org-alias>

# run the full test suite with coverage
sf apex run test --target-org <your-org-alias> \
  --tests EnrolmentTriggerTest --tests CohortServicesTest \
  --code-coverage --result-format human --wait 10
# expected: 10/10 Pass, class coverage 79-100%

# commit, push and tag the release
git add .
git commit -m "Week 3: LWC console, tests, permission set"
git push
git tag v1.0
git push --tags

# optional: validate against a second org (deploys nothing)
sf org login web --alias targetorg
sf project deploy start --source-dir force-app --target-org targetorg \
  --test-level RunLocalTests --dry-run
```

UAT script — prove every story works

ID	Story	Test step	Expected result
UAT-01	US-01	Create a Course and a Cohort linked to it	Both save; the Cohort shows the Course
UAT-02	US-01	Set Cohort End Date before Start Date	Validation error shown
UAT-03	US-03	Create one enrolment on a capacity-3 cohort	Enrolled Count = 1, Seats Remaining = 2
UAT-04	US-04	Enrol a 4th student on a capacity-3 cohort	Save blocked with the custom capacity message
UAT-05	US-05	Enrol the same student twice on one cohort	Save blocked (duplicate key)
UAT-06	US-06	Change an enrolment Status to Confirmed	Email sent to the student; follow-up Task created
UAT-07	US-07	Fill the final seat on a cohort	Cohort Status becomes Full automatically
UAT-08	US-08	Run the Quick Enrol screen flow end to end	Enrolment created; confirmation screen shown
UAT-09	US-09	Cohort start date = today+3; run the scheduled flow	Reminder Task for confirmed enrolments only
UAT-10	US-10	Enrolment Console: filter, select, enrol	Toast shown; seat count refreshes; record created
UAT-11	US-11	Ask the agent “which Advanced cohorts have seats?”	Live cohorts with dates and seats remaining
UAT-12	US-12	Ask the agent for status with a known email	Cohort, start date and status returned
UAT-13	US-13	Check the student’s Contact record	Total Enrolments reflects active enrolments
UAT-14	US-14	Run all Apex tests	All pass; coverage ≥ 75%
UAT-15	US-14	Deploy to a second org; repeat UAT-03 & UAT-10	Both behave identically

Appendix A · Fixes discovered in the reference deployment (expect these)

Symptom	Root cause	Fix
Contact inserts fail in tests with DUPLICATES_DETECTED	The org has an active Contact duplicate rule	TestDataFactory inserts contacts with Database.DMLOptions.DuplicateRuleHeader.allowSave = true
QueryException “secure query included inaccessible field”	WITH SECURITY_ENFORCED enforces the running user’s FLS; API-created fields grant FLS to nobody	Assign the Enrolment_Console_User permission set before running tests
CANNOT_EXECUTE_FLOW_TRIGGER on confirmed enrolments in tests	The flow’s Send Email action failed (deliverability/limits) and killed the save	Fault path on the email action so a failed email never blocks the enrolment
Bulk test expected exactly 150 successes, got fewer	Partial-save retries + the unique key make “exactly capacity” non-deterministic in one 200-record batch	Assert the true invariant: the cohort can never exceed capacity, and the overflow records are always blocked

Appendix B · Everyday commands (copy & paste)

```
sf org open --target-org <your-org-alias> # open the org in a browser
sf project retrieve start --manifest manifest/package.xml \
  --target-org <your-org-alias> # pull clicks-built changes into source
sf project deploy start --source-dir force-app/main/default/classes \
  --target-org <your-org-alias> # redeploy just the Apex
sf apex run --file scripts/apex/seed.apex --target-org <your-org-alias>
sf apex tail log --target-org <your-org-alias> # stream debug logs
git status ; git diff # see what changed
git checkout -b feature/waitlist # branch for a change
```

Appendix C · What the finished project contains

Type	Components
Custom Objects	Course__c, Cohort__c, Enrolment__c (+ Contact.Total_Enrolments__c)
Validation Rules	End_Date_After_Start_Date, Payment_Not_Above_Fee
Flows (Active)	Enrolment_Confirmed_Actions, Quick_Enrol_Student, Cohort_Start_Reminder
Apex	EnrolmentTrigger, EnrolmentTriggerHandler, StudentSummaryQueueable, CohortAvailabilityService, EnrolmentStatusService, CohortController, EnrolmentController, TestDataFactory, EnrolmentTriggerTest, CohortServicesTest
LWC	cohortExplorer, enrolmentPanel, enrolmentConsole
Pages / Tabs	Enrolment_Console app page · Course, Cohort, Enrolment tabs
Security	Enrolment_Console_User permission set
Scripts	scripts/apex/seed.apex · manifest/package.xml